

Artificial Intelligence Exam Notes

CT 710 (BEI, IV-I) · CT 653 (BCT, III-II) · CT 78506 (BEX, IV-II) · TU, IOE · tiers over 41 papers, 2069–2082 BS

6. Machine Learning

6 hrs · asked in **40 of 41 papers** · 6 syllabus sub-topics · **436 marks, 12.9% of the archive** · **74 numerical (17%) + 362 theory** · the ID3 data tables, the GA generations and the fuzzy arithmetic are worked in *Ch6 – Machine Learning – Solved Problems*.

Read this first:

- **Two topics carry this chapter, and both are drawings.** Genetic algorithm **TOP 21/41** and fuzzy logic **TOP 16/41**. Between them they are asked in **31 of the 41 papers** and they take roughly two thirds of the chapter's marks. Learn the GA flowchart and the fuzzy-inference block diagram cold; everything else here is a bonus.
 - **The examiner pairs them with each other.** 82 Ka, 82 Bh and 81 Ba all ask fuzzy logic and GA in the *same* question. Answering one well and the other badly costs the whole 8 marks.
 - **Third is supervised Vs unsupervised** **TOP 12/41**, always with the rider “*clustering falls under which category? Justify, stating algorithms*”. Name **K-means** and **hierarchical clustering** or the justification scores nothing.
 - **ID3 is the only real arithmetic in the chapter** **HOT 6/41**, and it is always the same shape: entropy of the whole set, entropy of each attribute's split, gain = difference, pick the largest. Three different data tables appear, all worked in the companion.
 - **The syllabus line says “fuzzy learning” and no paper has ever asked that.** Every one of the sixteen asks the **fuzzy inference system** — membership functions, rules, fuzzification, defuzzification. §6.7 teaches the inference system and says where the two names meet.
 - Reliable short notes, each a fixed list you can write in four minutes: **GA** 3 4, **fuzzy logic** 3 4, **Boltzmann machine** 4, **supervised Vs unsupervised** 4, **mutation Vs crossover** 3, **induction Vs deduction** 3.
-

6.1 What machine learning is, and the learning framework

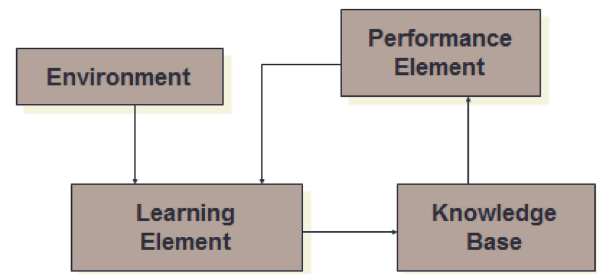
What is machine learning? Explain the learning framework with a block diagram. 1 6 2+6

PIN 2/41 78 Ba 2+6, 69 Bh 6 · the definition alone opens eleven more papers, always as the 1 or 2 lead-in to a GA, fuzzy or Boltzmann question (80 Ash 1 asks nothing else) · so learn the definition as one sentence and the framework as four boxes

Definition

- Machine learning is the branch of AI in which a program improves its performance on a task from experience, without being explicitly programmed for that task.
- Tom Mitchell's form, worth quoting: a program **learns** from experience E w.r.t. task T and measure P if **its performance at T , measured by P , improves with E** .
- In AI terms: the agent **observes data, builds a model** of it, and uses that model as both a hypothesis about the world and a program that solves problems.
- Three things learning always does**, and the three-word answer to "what is learning":
 - Change** – the learner itself is altered.
 - Generalization** – performance improves on *similar* tasks, not only on the ones seen.
 - Improvement** – and the design must stop changes that make performance worse.
- Why an intelligent agent needs it** (asked as "learning is an essential characteristic of intelligent agents, comment", Ch 1): a designer cannot enumerate every situation; the environment changes after deployment; and some tasks have no known algorithm, only examples.

The learning framework: four components



Environment feeds the learning element; the learning element writes the knowledge base; the performance element uses it and reports back.

- 1. Environment** – supplies the information. Judge it on:
 - Level:** high level = abstract, covers a broad class of problems; low level = detailed, covers one problem.
 - Quality:** noise free, reliable, ordered.
- 2. Learning element** – turns that information into knowledge. Rote learning, learning by example, by analogy, explanation-based (§6.2).
- 3. Knowledge base** – stores what was learnt. Must be **expressive, modifiable** and **extendible** (holds meta-knowledge, so its own structure can change).
- 4. Performance element** – uses the knowledge to act, measures how well it did, and **feeds the error back** to the learning element. That feedback arrow is the whole point of the diagram: draw it.

6.2 Ways of learning, and induction Vs deduction

What are the different ways / methods of learning? Describe each. 8 3+5

PIN 2/41 71 Bh 8, 81 Ba 3+5 · 81 Ba asks it as the 3 half of a GA question, so give the list, not essays · 78 Ch 2 asks the narrower “algorithms inspired by biological phenomena and social behaviour”: genetic algorithms, neural networks, swarm intelligence (ant colony, particle swarm), artificial immune systems

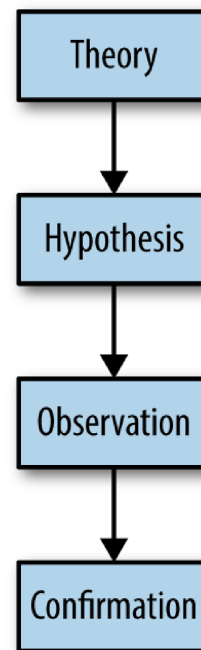
The five ways, easiest to hardest

- **1. Rote learning** – memorise and recall.
 - Solved problem → store → look up next time.
 - No inference at all: nothing new is derived.
 - **Only worth it if retrieval is cheaper than recomputing.**
 - Eg Samuel’s checkers player, which stored board positions it had seen.
- **2. Learning by example (inductive)** – specific cases → general rule. Eg decision trees, ID3 (§6.5).
- **3. Learning by analogy** – transfer a solution from a known similar problem (§6.4).
- **4. Explanation-based learning (EBL)** – use existing domain knowledge to *explain* one example, then generalise the explanation into a rule.
 - Learns from **a single example**, because the prior knowledge does the work the extra examples would have done.
 - Eg see one cup once, know what makes it a cup, recognise every cup after.
- **5. Learning by deduction** – general rules → specific facts, or new general rules from old ones. $A = B$, $B = C$, so $A = C$.

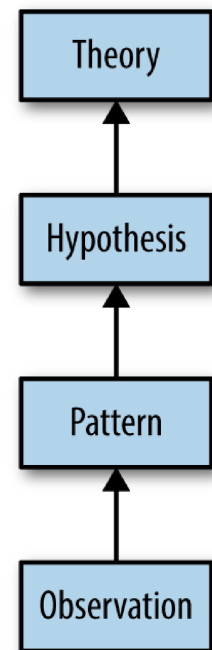
Induction Vs deduction

asked as a short note by 73 Bh 3.

Deductive Reasoning



Inductive Reasoning



Read the left ladder downwards and the right one upwards: that direction is the whole distinction.

	Inductive	Deductive
Direction	observation → hypothesis	hypothesis → observation
Goes from	specific	general
Produces	a general rule from raw data	a specific fact from known rules
Truth	not guaranteed , only probable	guaranteed if the premises hold
Reasoning	probabilistic	logical (Ch 4)
Example	1000 white swans → “swans are white”	all men are mortal; Socrates is a man; so Socrates is mortal

- **The one-line answer:** induction generalises and can be wrong; deduction specialises and cannot.
- Both appear in §6.2’s list of ways of learning, and the papers ask them as a 3 short note, so three rows of that table plus this line is a full answer.
- **Where each one lives in this subject:** deduction is Chapter 4 (resolution, forward and backward chaining, all truth-preserving); induction is this chapter (ID3, §6.5) and every learning algorithm in it.
- **Abduction** is worth one extra line if you have room: reasoning from an observation to its *most likely cause*. Medical diagnosis. Also not truth-preserving.

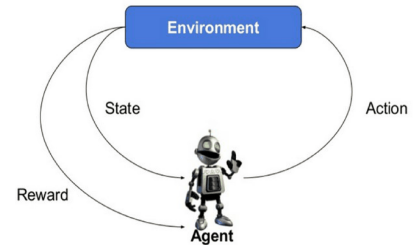
6.3 Supervised, unsupervised and reinforcement learning

Differentiate supervised and unsupervised learning. Clustering falls under which category? Justify, stating algorithms. 4 5 2+4 4+4 2+6

TOP 12/41 74 Ma 2+4, 72 Ash 2+4, 74 Bh 4+4, 79 Jth 4+4, 76 Bh 5, 78 Po 2+6, 69 Po 4, 74 Bh 4, 78 Ba 4, 77 Ch 4, 81 Ash 4, 75 Bh 4 · **third-most-asked topic in the whole subject** · two riders recur: “clustering falls under which?” (74 Ma, 72 Ash) and “learning is essential for intelligent agents, comment” (74 Bh, 79 Jth) · 78 Po wants all three types with examples

	Supervised	Unsupervised
Training data	labelled: every input carries its correct output	unlabelled: inputs only
Aim	learn the input → output mapping, then predict for new inputs	find hidden structure, groups and patterns in the data
Feedback	direct: error = predicted – actual, at every sample	none; the model is judged only by the structure it finds
Tasks	classification, regression	clustering, association
Algorithms	linear + logistic regression, SVM, naive Bayes, KNN, decision tree, back-propagation ANN	K-means, hierarchical (agglomerative), DBSCAN, Apriori, SOM
Human effort	high: someone has to label everything	low
Accuracy	higher, and measurable	lower, and hard to measure
Example	spam filter trained on mails already marked spam / not spam	segment customers by purchase behaviour, with no idea of the segments beforehand

Reinforcement learning



- **Feedback-based:** the agent acts, the environment returns a **reward** or a **penalty**, and the agent adjusts.
- No labelled data and no supervisor. The agent **discovers** which actions pay by trying them.
- Goal = maximise **cumulative** reward over the long run, not the next reward.
- **Four elements to name** (78 Po):
 - **Policy** – state → action mapping.
 - **Reward signal** – one number per step, defines the goal.
 - **Value function** – total reward expected from a state: good in the long run, where reward is good in the short run.
 - **Model** of the environment (optional).
- Example: a robot that gets +10 for reaching the goal square and –50 for falling in a pit. Also game playing, robot navigation, self-driving control.
- **Semi-supervised** is the fourth type worth one line: a small labelled set plus a large unlabelled one, used when labelling everything is too costly.

The rider: which category is clustering? (74 Ma, 72 Ash, 4 of the 6)

- **Clustering is unsupervised**, because the data carries **no class label** – the algorithm both invents the groups and assigns points to them.
- **Justify by naming algorithms**, which is what the question asks for and where the marks are:
 - **K-means** – choose k , assign every point to the nearest centroid, recompute the centroids, repeat until nothing moves. Nobody ever tells it a right answer.
 - **Hierarchical (agglomerative)** – start with every point its own cluster, repeatedly merge the two nearest, stop at k clusters or one.
 - **DBSCAN** – groups by density, and finds the number of clusters itself.
 - **SOM / Kohonen** (Ch 7) is a clustering network, and is unsupervised for exactly the same reason.
- **Contrast with classification**, which draws the same picture but is supervised: the classes exist *before* training and every training point already carries one.

Deciding the type of any example the examiner invents

What does the data carry?

Then it is

a correct output for every input	supervised
nothing but inputs	unsupervised
a few labelled, most not	semi-supervised
no outputs, but a reward after acting	reinforcement

- Worked on the examples the papers use: spam filter = supervised (mails already marked); customer segmentation = unsupervised; game-playing agent = reinforcement; handwriting recognition trained on labelled digits = supervised; market-basket association = unsupervised.
- **One sentence that is always worth writing:** supervised learning is told the answer and learns the mapping; unsupervised learning is told nothing and learns the structure; reinforcement learning is told only whether it did well, and learns the policy.

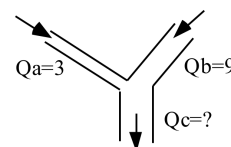
6.4 Learning by analogy

What is learning by analogy? Give an example, with a semantic network. 3 2+6 1+7

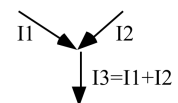
PIN 3/41 71 Bh 1+7, 76 Ba 2+6, 71 Ma 3 · 71 Bh demands the semantic network, so the answer is a *drawing plus the mapping table* · 71 Ma asks it as *analogy Vs inductive learning*

- **Learning by analogy = acquiring knowledge about a new entity by transferring it from a known, similar entity.**
- It carries a solution **across domains**: find the analogous states and operators in the two domains, then reuse the old solution.
- **One example is enough**, which is what makes it different from inductive learning (which needs many).
- **The five steps** (write these for a 6 or 7 mark answer):
 - **1. Recognition** – notice the new problem resembles a past one.
 - **2. Access and recall** – the resemblance indexes memory; retrieve one or more candidate analogues.
 - **3. Selection and mapping** – pick the relevant parts and map them from the **base** (known) domain to the **target** (new) one.
 - **4. Extension** – modify and extend the mapped solution to fit the target.
 - **5. Validation and generalization** – test it; if it holds, form a general rule covering both situations.
- **Analogy Vs induction** (71 Ma 3): analogy maps *one* case onto *one* case and reasons by similarity; induction generalises *many* cases into a rule. Analogy transfers, induction abstracts.

First example: hydraulics and Kirchhoff



Simple Hydraulics Problem



Kirchoff's First Law

- Flow into a junction equals flow out; current into a node equals current out. Knowing $Q_a = 3$, $Q_b = 9$ gives Q_c the same way $I_3 = I_1 + I_2$ does.
- **The mapping:** pressure drop $\leftrightarrow$ voltage drop, pipe $\leftrightarrow$ wire, flow rate $\leftrightarrow$ current, junction $\leftrightarrow$ node.
- So a student who knows Kirchhoff's law can solve the hydraulics problem **without being taught hydraulics**. That sentence is the definition made concrete, and it is what the examiner wants.

- **Second example: the atom and the solar system**, the one to draw as a **semantic net** for 71 Bh: two small nets, one arc list mapped onto the other.
 - Base net: Sun --greater-mass--> Earth,
Sun --attracts--> Earth,
Earth --revolves-around--> Sun.
 - Target net: Nucleus --greater-mass--> Electron,
Nucleus --attracts--> Electron, and the arc analogy **supplies**: Electron --revolves-around--> Nucleus.
 - Draw the two nets side by side and a **dashed arrow between the matching nodes**: Sun ↔ Nucleus, Earth ↔ Electron. That dashed mapping is the answer.

The mapping table, which is the marked half

Role	Base: solar system	Target: atom
Heavy central body	Sun	Nucleus
Light orbiting body	Earth	Electron
Relation known	attracts, greater mass	attracts, greater mass
Relation transferred	Earth revolves around Sun	Electron revolves around Nucleus

- The last row is the whole point: it is **not observed, it is carried across** by the analogy, and then validated.
- Say in one line what analogy risks: the mapping may not hold (electrons do not really orbit like planets), so step 5 validation is not optional. Naming that limitation reliably earns a mark.

6.5 Inductive learning, decision trees and ID3

Define inductive learning. How is the best attribute selected in a decision tree? Explain ID3 with an example. 8 1+7 2+6 2+7

HOT 6/41 74 Bh 2+7, 74 Ma 8, 72 Ash 8, 79 Jth 1+7, 77 Ch 8, 74 Bh 2+6 · five of the six print a data table and want the root attribute computed, so this section is theory only – all three tables are worked in the companion · 77 Ch disguises it as the *mushroom island* story

Inductive learning, and the tree

- Inductive learning = drawing a general hypothesis from a set of specific input–output examples, chosen so that it classifies the examples well *and* generalises to new ones.
- A decision tree is the standard hypothesis form:
 - each **internal node** tests one attribute,
 - each **branch** is one value of that attribute,
 - each **leaf** carries a class label.
- To classify, walk root to leaf and read off the label.
- ID3 (Iterative Dichotomiser 3, Quinlan) builds that tree **greedily, top down**: at every node it puts the attribute that separates the classes best, and never goes back.

The two formulas – memorise these

$$H(S) = - \sum_i p_i \log_2 p_i$$

entropy of set S : p_i is the fraction of S in class i . $H = 0$ when S is pure, $H = 1$ when a two-class set is split 50/50.

$$\text{Gain}(S, A) = H(S) - \sum_{v \in A} \frac{|S_v|}{|S|} H(S_v)$$

the second term is the *weighted average* entropy after splitting on A . Gain is the uncertainty removed.

- **Highest gain = lowest weighted entropy = best attribute.**
The two rules are the same rule.

Why it works, in one figure

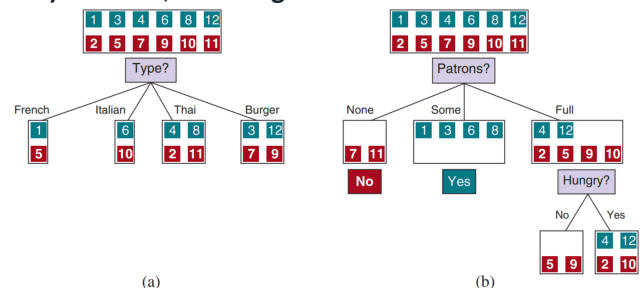


Figure 19.4 Splitting the examples by testing on attributes. At each node we show the positive (light boxes) and negative (dark boxes) examples remaining. (a) Splitting on *Type* brings us no nearer to distinguishing between positive and negative examples. (b) Splitting on *Patrons* does a good job of separating positive and negative examples. After splitting on *Patrons*, *Hungry* is a fairly good second test.

Splitting on *Type* leaves every branch as mixed as the parent: gain 0, useless. Splitting on *Patrons* produces two pure branches and one mixed one: high gain. That comparison is the answer to “how is the best attribute selected”.

The algorithm, as five steps

- 1. Compute $H(S)$ for the current set.
- 2. For each unused attribute A , split S by its values and compute the weighted entropy of the split.
- 3. $\text{Gain}(S, A) = H(S) -$ that weighted entropy.
- 4. Make the attribute with the **largest gain** the node, and branch on its values.
- 5. Recurse on each branch with the remaining attributes. Stop a branch when: it is **pure** (leaf = that class); **no attributes are left** (leaf = majority class); or it is **empty** (leaf = parent's majority).
- Three arithmetic checks that save marks:** H of a pure branch is 0, not 1; H of an even two-class split is exactly 1; and every gain must be ≥ 0 , so a negative one is an error in your weighted sum.

Characteristics to list if asked

- Greedy, so **no guarantee of the optimal tree** – it can settle in a local optimum.
- Produces small trees, but not always the smallest.
- Overfits** the training data; prefer smaller trees, and prune.
- Awkward on **continuous** attributes: too many possible split points.
- Handles noise and missing values badly. C4.5 fixes all four.

6.6 Genetic algorithm

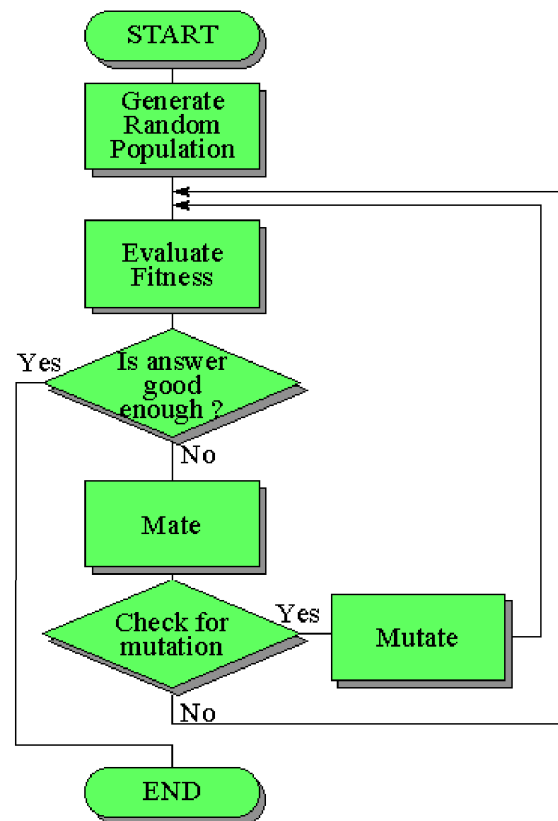
What is a genetic algorithm? Explain all its steps with a block diagram, and its operators. 3 4 5 7 8

10 2+8

TOP 21/41 69 Po 10, 71 Ma 2+8, 72 Ma 2+6+2, 75 Ba 8, 77 Ch 8, 81 Bh 2+6, 79 Ch 2+6, 80 Ba 8, 76 Bh 5, 81 Ba 3+5, 72 Ma 7, 82 Ka 2+6, 82 Bh 2+6, 81 Ash 3+5, 81 Ch 7, 78 Ch 2+6, 74 Bh 4, 73 Bh 3, 70 Bh 2+6, 74 Bh 4+4, 75 Bh 2+8 · the single most-asked topic in the chapter and the second most-asked in the subject · it has been set in every year from 2069 to 2082 · the wording rotates but the answer never does: steps + block diagram + the three operators + a worked example

Definition and idea

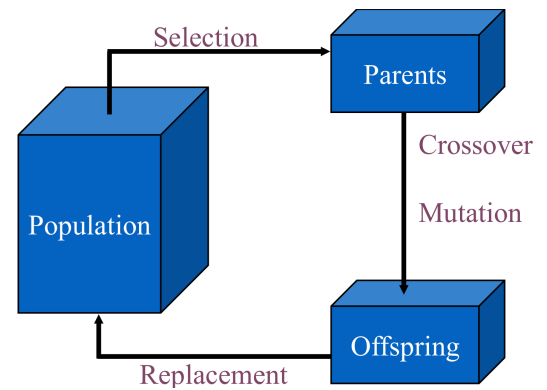
- A genetic algorithm is a search and optimization technique that imitates natural selection: it keeps a population of candidate solutions and evolves it, generation by generation, towards better ones.
- Darwin in one line: **select the best, discard the rest.**
- Evolutionary computing** (asked by name in 79 Ch) is the family; GA is its best-known member, alongside genetic programming, evolutionary strategies and swarm methods.
- Formally a variant of **stochastic beam search**: many states kept at once, successors generated by combining pairs rather than mutating one.
- Vocabulary** – the examiner marks these:
 - Gene** – one position in the string; controls one characteristic.
 - Chromosome** (individual) – a chain of genes = **one candidate solution**, usually a binary string.
 - Population** – the set of chromosomes alive in one generation.
 - Fitness function** – scores a chromosome. **Problem dependent; it is the only thing that knows what “good” means.**
 - Generation** – one full cycle of the loop.



The flowchart 81 Ba asks for by name. Draw START → generate random population → evaluate fitness → good enough? → (no) select and mate → mutate → back to evaluate; (yes) END.

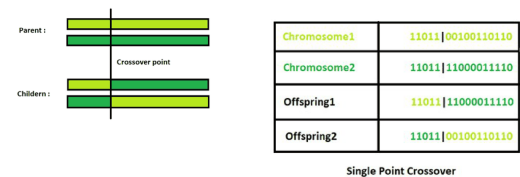
The five steps, which are also the five operators

- **1. Initialization** – generate the starting population at random.
 - Encode a solution as a string first (**chromosome encoding**); binary is usual, but integer and permutation encodings exist.
 - Population size trades diversity against run time. Seed it around a promising region if you have one.
- **2. Fitness evaluation** – score every chromosome with the fitness function. Infeasible chromosomes get fitness 0.
- **3. Selection** – choose parents, with probability rising with fitness.
 - **Roulette wheel**: slice proportional to fitness. **Tournament**: pick k at random, take the best. **Rank**: order them, select on rank. **Elitism**: copy the best one or two through untouched.
 - **Keep a few weak ones on purpose** – they hold the diversity the next crossover needs.
- **4. Crossover (recombination)** – cut two parents and swap the tails, so each child inherits from both. Probability typically 0.6 to 0.9.
- **5. Mutation** – flip a random gene with a small probability, typically 0.001 to 0.01.
- **6. Replacement, then termination** – offspring replace the weakest members; stop on a **fixed number of generations**, a **fitness threshold**, or when the population has **converged** (no improvement for several generations).

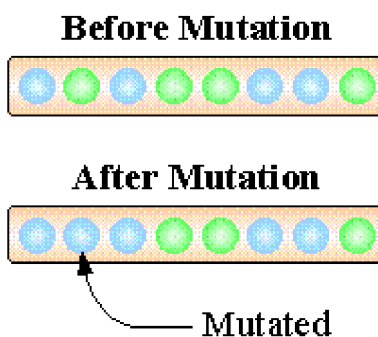


The same algorithm as one cycle, which is the “block diagram” most papers want: population → selection → parents → crossover, mutation → offspring → replacement → population.

Crossover, drawn



Single-point: one cut, tails swapped. **Two-point**: two cuts, the middle segment swapped. **Uniform**: each gene taken from either parent by a coin toss.



Mutation Vs crossover (73 Bh 3)

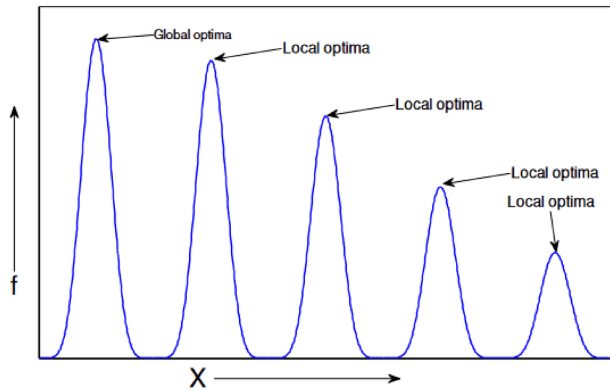
	Crossover	Mutation
Parents	two	one
Does	exchanges segments between two chromosomes	flips a gene in one chromosome
Rate	high, 0.6–0.9	low, 0.001–0.01
Role	exploitation : combines what already works	exploration : introduces what is missing
If absent	population stagnates as copies of a few parents	search sticks in a local optimum

Mutation flips one gene of one child.

Why mutation matters (81 Ba asks exactly this)

- Crossover only **recombines genes already in the population**. If every chromosome carries a 0 in some position, no amount of crossover will ever produce a 1 there.
- Mutation is the **only source of new genetic material**. It keeps diversity up and stops the search converging early on a **local optimum**.
- So the rate must be *small*: too high and the search degenerates into a random walk, losing the good solutions selection just found.

When shall we use a genetic algorithm? (81 Ash, 80 Ch)



One global and four local optima. A hill-climber stops on whichever bump it started under; a population searches all five at once.

- The search space has **many local optima** and gradient methods get trapped.
- The objective function is **not smooth**, not differentiable, or is only available as a black box, so no derivative method applies.
- The number of parameters is **very large** and exhaustive search is impossible.
- The objective is **noisy or stochastic**.
- A **good-enough** answer is acceptable: GA gives no optimality guarantee.
- **Do not use it** when an exact algorithm exists, when a single evaluation of the fitness function is expensive, or when the answer must be provably optimal.

Where it is applied (72 Ma 2)

- Optimization: travelling salesman, knapsack, job-shop scheduling, timetabling.
- Engineering design, antenna and circuit layout, PID controller tuning.
- Neural network weight and architecture search; feature selection in ML.
- Robotics path planning, vehicle routing, bioinformatics (gene sequencing).
- **Why the study of genes belongs in AI (74 Bh 4):** evolution is a **proven search procedure** that solved design problems no designer could; GA borrows the mechanism, not the biology.

Advantages and limitations

- ✓ Simple concept; works on the **encoded** parameters, so it needs no derivative and no auxiliary information, only a fitness score.
- ✓ Searches many points at once, so it escapes local optima; inherently **parallel**; the answer improves with run time and can be stopped anywhere.
- × No guarantee of the global optimum; × many fitness evaluations, so slow when each is costly; × the encoding, population size and rates all need tuning; × can converge prematurely if diversity is lost.

6.7 Fuzzy logic and the fuzzy inference system

What is fuzzy logic? When do we use it? Explain fuzzy inference / the steps, with an example. 2 3

4 8 2+6 3+7 1+2+5

TOP 16/41 69 Bh 4, 70 Ma 2+6, 80 Ash 1+2+5, 73 Ma 3+7, 79 Ash 1+2+5, 80 Ch 2+5, 82 Ba 2+6, 80 Bh 8, 79 Bh 2+6, 81 Ba 4, 78 Po 4, 77 Ch 4, 81 Ash 4, 80 Ba 3, 82 Ka 2, 82 Bh 2 · the syllabus calls this “fuzzy learning” and no paper ever has: they ask fuzzy logic and fuzzy inference, which is reasoning, not learning · 73 Ma is the only one to name Mamdani · 82 Ka and 82 Bh bolt it onto a GA question

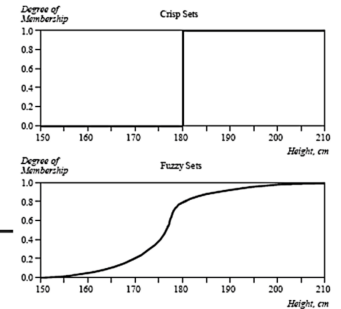
What it is

- Fuzzy logic is a many-valued logic in which a proposition has a degree of truth anywhere in $[0, 1]$, instead of only 0 or 1.
- It exists because human concepts have **no sharp boundary**: *tall, hot, slow, near, honest*. A crisp rule must answer *is 179 cm tall?* yes or no; a fuzzy one answers 0.78.
- Introduced by **Lotfi A. Zadeh**, UC Berkeley, **1965**.
- **Fuzzy set**: for a universe U , a fuzzy set A is the set of pairs $\{(x, \mu_A(x)) : x \in U\}$ where the **membership function**

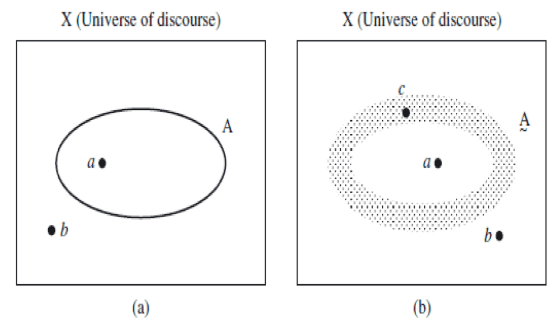
$$\mu_A(x) : U \rightarrow [0, 1]$$

- $\mu_A(x) = 1 \rightarrow x$ is fully in A ; $\mu_A(x) = 0 \rightarrow$ fully out;
 $0 < \mu_A(x) < 1 \rightarrow$ partly in.
- **Linguistic variable** – a variable whose values are words: Temperature = {very cold, cold, warm, hot, very hot}. Each word is a fuzzy set with its own membership curve, and the curves **overlap** on purpose.

Name	Height, cm	Degree of Membership	
		Crisp	Fuzzy
Chris	208	1	1.00
Mark	205	1	1.00
John	198	1	0.98
Tom	181	1	0.82
David	179	0	0.78
Mike	172	0	0.24
Bob	167	0	0.15
Steven	158	0	0.06
Bill	155	0	0.01
Peter	152	0	0.00



Crisp Vs fuzzy on the same data. Membership of *tall*: the crisp column steps from 0 to 1 at 180 cm, so Tom at 181 is tall and David at 179 is not. The fuzzy column grades them 0.82 and 0.78 – which is what a person would say. This one table answers “how does fuzzy logic differ from classical binary logic” (82 Bh) outright.



Diagrams for (a) crisp set boundary and (b) fuzzy set boundary.

The same point as sets: b is simply outside crisp A ; in fuzzy A the boundary is a band, so b and c have partial membership.

Fuzzy set operations

all defined pointwise on the membership values; worked on real numbers in the companion.

Operation	Membership
Union $A \cup B$	$\max(\mu_A, \mu_B)$
Intersection $A \cap B$	$\min(\mu_A, \mu_B)$
Complement A'	$1 - \mu_A$
Bold union (X-OR)	$\min(1, \mu_A + \mu_B)$
Bold intersection	$\max(0, \mu_A + \mu_B - 1)$
Equality	$\mu_A(x) = \mu_B(x)$ for every x

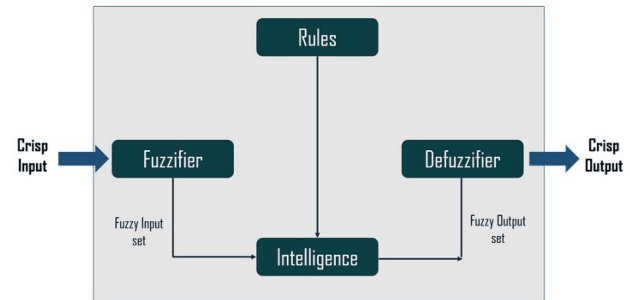
Fuzzy logic Vs probability

- **Not the same thing, and papers reward saying so.**
- *Ram's membership of "old" is 0.9* – Ram **is** 90 % old. A statement about **vagueness**, and it stays 0.9 after you meet him.
- $P(\text{Ram is old}) = 0.9$ – Ram either is or is not old and we do not know which. A statement about **ignorance**, and it collapses to 0 or 1 once you meet him.

The steps of “fuzzy learning” (80 Bh 8, 79 Bh 2+6, 69 Bh 4)

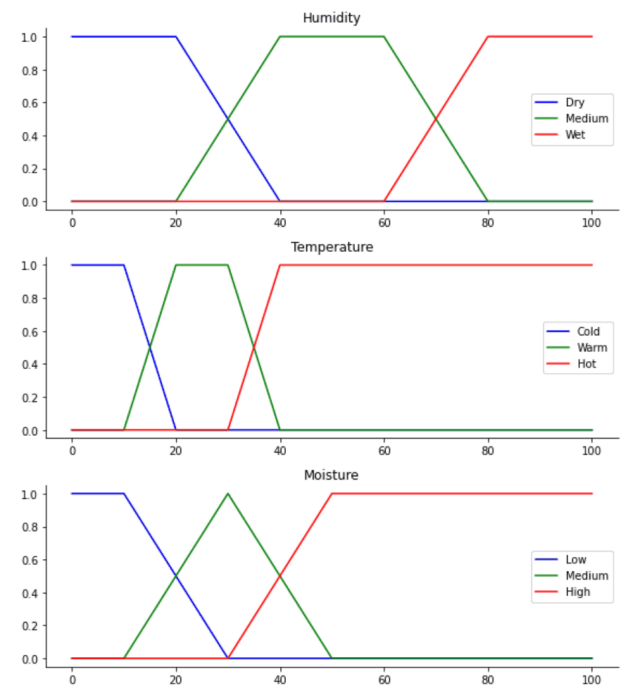
- **1. Define the linguistic variables and terms.** Temperature = {very cold, cold, warm, very warm, hot}.
- **2. Construct a membership function for each term.** Triangular and trapezoidal are normal; they must overlap.
- **3. Build the rule base** of IF-THEN rules, usually as a matrix of every input combination against the action to take.
- **4. Fuzzify:** convert the crisp readings into membership degrees.
- **5. Evaluate the rules** in the inference engine, and **aggregate** the results into one fuzzy output.
- **6. Defuzzify** into a crisp control value.
- **Worked end to end on a fan-speed controller in the companion, §6.N.12; §6.N.13** builds that controller's variables, membership functions and rule base from a blank sheet, which is what 80 Ch and 82 Ba ask for instead of a run.

Architecture of a fuzzy inference system



The block diagram 82 Ba and five other papers ask for: crisp input → fuzzifier → fuzzy input set → inference engine (driven by the rule base above it) → fuzzy output set → defuzzifier → crisp output.

- **Rule base** – the IF-THEN rules supplied by domain experts. *IF temperature is cold AND target is warm THEN heater is on.*
- **Fuzzification** – read the crisp sensor value off each membership curve to get a degree for every linguistic term.
- **Inference engine** – match the fuzzified inputs against the rules, compute each rule's **firing strength**, and combine the consequents.
- **Defuzzification** – collapse the aggregated fuzzy output back to one crisp number. **Centroid** (centre of gravity) is the usual method; also mean of maxima, bisector, weighted average.



Step 2 for three linguistic variables of a soil-moisture controller.

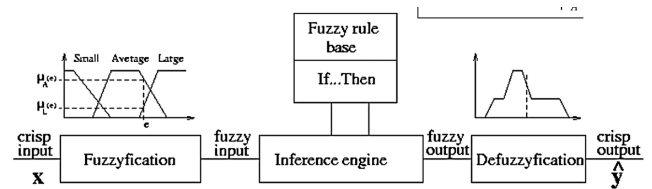
Note the overlaps: at 45 the humidity is *both* medium and wet, to some degree.

Mamdani fuzzy inference (73 Ma 3+7)

- Built by **Ebrahim Mamdani**, London, **1975**, to control a steam engine and boiler from rules given by human operators.
- Its six operations, in order: **determine the rules**, **fuzzify** the inputs, combine them to get each rule's **strength** (fuzzy AND = min), apply **implication** to clip the output set, **aggregate** the clipped sets, then **defuzzify**.
- **Mamdani Vs Sugeno** in one line: Mamdani's rule consequent is a *fuzzy set* and needs defuzzifying; Sugeno's is a *function of the inputs*, so the output is a weighted average and no defuzzification is needed. Mamdani is more intuitive, Sugeno cheaper.

Advantages

- Mimics human reasoning, so the rules are **readable and can be written by a domain expert** who knows no mathematics.
- Needs no exact mathematical model of the plant – decisive where one does not exist.
- **Robust**: imprecise, noisy or partly missing inputs still give a sensible output, so it degrades gracefully when a sensor fails.
- Simple, cheap to implement on small hardware; handles nonlinearity of arbitrary complexity.
- Proven in real products: washing machines, air conditioners, camera autofocus, anti-lock brakes, subway train braking.



The same architecture with the membership curves drawn on it, so “fuzzy input” and “fuzzy output” are visible as shapes rather than words.

Disadvantages, and when not to use it

- **No systematic design method.** Rules and membership functions come from expert opinion, so two designers get two systems.
- Accuracy depends entirely on the quality of that expert knowledge.
- Validation and verification need extensive hardware testing; there is no proof of stability the way there is for a classical controller.
- Not suited to problems needing high precision, nor to pattern recognition where a neural network learns the mapping from data.
- **Do not use it** when the input space does not map naturally to an output space, when a crisp model already works, or where common sense or a simple threshold would do.

6.8 Boltzmann machine

Define the Boltzmann machine. Explain it with a suitable algorithm. 4 1+7 2+8

PIN 3/41 70 Ma 1+7, 72 Ash 2+8, 76 Ba 4 · only three papers, but two of them put 7 or 8 marks on it · 70 Ma pairs it with the semantic network (Ch 5), 72 Ash with “what is machine learning”

- A Boltzmann machine is a stochastic, energy-based, recurrent neural network of symmetrically connected binary units that switch on and off at random, with a probability set by the network's energy.
- Invented by Hinton and Sejnowski, 1985. Named for the Boltzmann distribution it samples from.
- Structure:
 - Visible units V carry the data in and the reconstruction out.
 - Hidden units H capture features and dependencies.
 - Every pair is symmetrically connected, $w_{ij} = w_{ji}$, and there are **no self connections**.
 - **There is no output layer** – the same visible nodes that take the input return the reconstruction.
- State and energy. Each unit is 0 or 1; the network's energy is

$$E = - \sum_{i < j} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

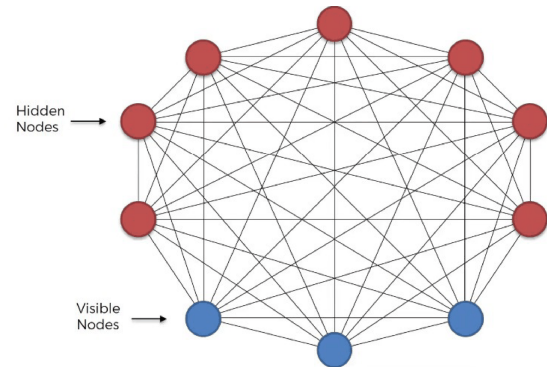
and the probability of a state falls exponentially with its energy:

$$p_i = \frac{e^{-\varepsilon_i/kT}}{\sum_j e^{-\varepsilon_j/kT}}$$

- So the machine **prefers low-energy states but can still climb out of one**, which is how it escapes local minima – the same idea as simulated annealing (Ch 3), with T lowered as it runs.

The learning algorithm, in six steps

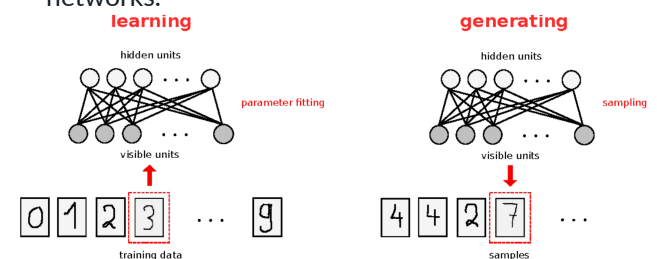
1. Initialise the weights randomly and set a high temperature T .
 2. **Positive phase (clamped)**: fix the visible units to a training vector, let the hidden units settle, record the correlation $\langle s_i s_j \rangle_{data}$.
 3. **Negative phase (free)**: release every unit, let the network run to equilibrium, record $\langle s_i s_j \rangle_{model}$.
 4. **Update** $\Delta w_{ij} = \eta (\langle s_i s_j \rangle_{data} - \langle s_i s_j \rangle_{model})$.
 5. Lower T a little (annealing) and repeat over the training set.
 6. Stop when the weights stop changing.
- **What the two phases mean**: the clamped phase measures how units co-fire when the world is showing them the truth, the free phase how they co-fire when the network is dreaming on its own. Learning pushes the second towards the first.



Visible nodes below, hidden above, every pair connected both ways. Compare with a feed-forward net: no layers, no direction, no output side.

Restricted Boltzmann machine (RBM)

- A full Boltzmann machine is **far too slow**: every unit depends on every other, so the weights must be updated one at a time.
- RBM **forbids connections inside a layer**. Visible-to-hidden only.
- Units within a layer are then independent given the other layer, so a whole layer updates **in parallel**. Trained by **contrastive divergence**.
- Used for dimensionality reduction, classification, regression, collaborative filtering (recommender systems), and as the building block of deep belief networks.



6.9 Every Chapter 6 question, grouped by the answer it wants

Every Chapter 6 question in the 41 papers, grouped by the answer it wants. **Bold** = regular exam, plain = back. Monospace = CT 710 (BEI), *italic* = CT 78506 (BEX), upright = CT 653 (BCT).

- **Genetic algorithm 21 papers** → §6.6, worked in the companion. The wordings, all one answer: *explain GA and its operators* (69 Po, 71 Ma, 72 Ma); *all steps with block diagram* (75 Ba, 77 Ch, 81 Bh, 80 Ba, 76 Bh, 78 Ch); *steps with a flowchart, and why mutation matters* (81 Ba); *working principle with examples* (72 Ma, 81 Ch); *when shall we use it* (81 Ash); *what is evolutionary computing* (79 Ch); *the algorithm of GA* (70 Bh); *justify that the study of genes is part of AI* (74 Bh); short notes (74 Bh, 73 Bh as mutation Vs crossover); and bolted to a fuzzy question in 82 Ka and 82 Bh.
- **Fuzzy logic and fuzzy inference 16 papers** → §6.7. *Fuzzy inference with an example* (70 Ma, 80 Ash); *Mamdani method* (73 Ma); *when do we use it, and the steps* (79 Ash, 80 Ch); *architecture and working mechanism* (82 Ba); *all the steps in fuzzy learning with a block diagram* (80 Bh, 79 Bh, which also asks *what is membership of an element*); *what is fuzzy learning, with a practical example* (69 Bh); and short notes at 81 Ba, 78 Po, 77 Ch, 81 Ash, 80 Ba.
- **Supervised Vs unsupervised, and the three types 12 papers** → §6.3. The *clustering rider* is in 74 Ma and 72 Ash; the “*learning is essential for intelligent agents*” rider in 74 Bh and 79 Jth (answered in Ch 1); all three types with examples in 78 Po.
- **ID3 and the best attribute 6 papers** → §6.5 for the method, companion for all three data tables: *play golf* (74 Bh, 74 Ma, 72 Ash), *play cricket* (79 Jth), *poisonous mushrooms* (77 Ch). 74 Bh asks the ID3 process without a table.
- **Ways of learning 2 papers** → §6.2 (71 Bh 8, 81 Ba 3). *Induction Vs deduction* as a short note at 73 Bh 3. *Algorithms inspired by biology and social behaviour* at 78 Ch 2.
- **Learning framework 2 papers** → §6.1 (78 Ba, 69 Bh). *Define machine learning* alone at 80 Ash 1, and as the lead-in to eleven other questions.
- **Learning by analogy 3 papers** → §6.4. 71 Bh wants it *with a semantic network*; 71 Ma wants *analogy Vs inductive learning* 3.
- **Boltzmann machine 3 papers** → §6.8 (70 Ma, 72 Ash, 76 Ba).
- **Swarm intelligence** 2 verified/added – asked once, 75 Bh, and **no lecture deck in this subject mentions it**. Answer: a family of optimization methods in which many simple agents following local rules produce intelligent group behaviour with no central control – ant colony optimization (pheromone trails, shortest path), particle swarm optimization (each particle steers towards its own best and the swarm’s best), bee colony, flocking. Like GA it is population based and nature inspired; unlike GA the individuals **are not replaced, they move**. The other half of that question, the GA fitness numerical, is worked in the companion.
- **Reinforcement learning in detail** – named by 78 Po only, and the one lecture deck that covers it is PS Sir’s. §6.3 carries the four elements (policy, reward signal, value function, model).
- **Not in this chapter:** *neural networks* appear on the syllabus line for Chapter 6 but every paper sets them under Applications of AI – perceptron, back-propagation, Hopfield, Kohonen and Hebbian learning are all in **Ch 7**. *Learning in a neural network* short notes (69 Po, 74 Bh, 78 Ba, 77 Ch, 81 Ash) are answered there.

Source notes

- Spine: BA Sir CH-06 Machine Learning, PS Sir 6. Machine Learning, SG Sir AIChapter_6, and Insights on Artificial Intelligence ch 6 for the worked method.
- **Two corrections carried into the companion.** Insights p167 prints $\mu_{A'}(30) = 0.5$ for a complement whose correct value is 0.6, and p166 labels a fuzzified pair (10, 0.3) when the element is 9, not 10. Both are shown correctly there, with a note.
- **Reinforcement learning and Mamdani inference are in PS Sir’s deck only**, which is labelled *old syllabus*. Both are on the current CT 710 syllabus by way of the papers, so both are kept, checked against the standard sources.
- Swarm intelligence is verified/added – researched and written for 75 Bh, because no deck in Notes\ contains the word.

Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

Short	Means	Short	Means
AI	artificial intelligence	ANI	artificial narrow intelligence
AGI	artificial general intelligence	ASI	artificial super intelligence
KR	knowledge representation	KRR	knowledge representation and reasoning
PEAS	performance measure, environment, actuators, sensors	GPS	General Problem Solver
NLP	natural language processing	NLU	natural language understanding
NLG	natural language generation	CAPTCHA	completely automated public Turing test to tell computers and humans apart
ML	machine learning	ANN, NN	artificial neural network
MLP	multilayer perceptron	BP	back-propagation
SOM	self-organizing map	GA	genetic algorithm
FIS	fuzzy inference system	ID3	Iterative Dichotomiser 3
CSP	constraint satisfaction problem	DFS	depth first search
BFS	breadth first search	DLS	depth limited search
IDS	iterative deepening search	UCS	uniform cost search
CNF	conjunctive normal form	WFF	well-formed formula
FOL	first-order logic	FOPL	first-order predicate logic
RRS	resolution refutation system	BBN	Bayesian belief network

Short	Means	Short	Means
ES	expert system	KB	knowledge base
CD	conceptual dependency	LISP	list processing (the language)
MOP	memory organization packet	isa	the class-to-class link of a semantic net (<i>instance</i> is the individual-to-class one)
attr(s)	attribute(s)	repo	repository
freq	frequency	config	configuration
info	information	no.	number
Eg	example	Vs	compare / differentiate
w/, w/o	with, without	+Fig	draw the figure
→	leads to, therefore	=	is, equals

Reading the year tags and chips

Exam-paper month codes: Ba = Baishakh · Jth = Jestha · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Po = Poush · Ma = Magh · Ch = Chaitra. Years are Bikram Sambat, so 80 Ch is 2080 Chaitra.

Bold year tags are Regular exams, plain are Back exams. A header printing “Regular / Back” in one box counts as Regular.

Three course codes, three faces. Plain = CT 653 (BCT, III-II, 26 papers) · monospace = CT 710 (BEI, IV-I, 7 papers) · *italic* = CT 78506 (BEX, IV-II, 8 papers). **The same year code names different papers under different codes:** 77 Ch and 77 Ch are two different exams, and only the font tells them apart.

Tier chips over all 41 papers, 2069–2082 BS: **TOP /41** = topic asked in **8 or more** papers · **HOT /41** = **4 to 7** papers · **PIN /41** = **1 to 3** papers but worth ≥ 6 marks.

Mark chips such as **2+6** are the marks that question actually carried. **syllabus, no PYQ yet** marks a syllabus topic never yet examined but written up anyway. **verified/added** marks anything researched and added that no source note in this folder covers.